

1 Construction of the Code

1.1 Understanding quantum Hamming codes

Taking a look at the smaller $[[7, 1, 3]]$ code, we first look at the $[7, 4, 3]$ classical Hamming code, link: <https://homepages.math.uic.edu/lee/s08/handouts/Hamming.pdf>.

The sequence $x_0x_1x_2x_3$ is encoded with parity check bits at power of 2 locations as $p_0p_1x_0p_2x_1x_2x_3$. The stabilizers for the classical code are

$$S_0 = p_0 \oplus x_0 \oplus x_1 \oplus x_3$$

$$S_1 = p_1 \oplus x_0 \oplus x_2 \oplus x_3$$

$$S_2 = p_2 \oplus x_1 \oplus x_2 \oplus x_3$$

The syndrome measurement gives the location of the error. So the parity check matrix is

$$H = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

Which admits 3 weight-4 stabilizers. To extend this to a quantum code, we separate the X and Z stabilizers, using a parity matrix for each of the same form as in the classical code. This will leave $7 - 2(3) = 1$ logical qubit, so this is the smallest quantum Hamming code. The stabilizers are:

$$S = \begin{pmatrix} X & X & X & X & I & I & I \\ X & X & I & I & X & X & I \\ X & I & X & I & X & I & X \\ Z & Z & Z & Z & I & I & I \\ Z & Z & I & I & Z & Z & I \\ Z & I & Z & I & Z & I & Z \end{pmatrix}$$

Then valid logical operators commute with all stabilizers, but anticommute with each other. The most common choice is

$$\bar{X} = XXXXXXXX \\ \bar{Z} = ZZZZZZZZ$$

But There are other valid choices such as: which I find in logstab.ipynb

$$\bar{X} = IXXIXII \\ \bar{Z} = IZZIZII$$

1.2 The $[[15, 7, 3]]$ code

Then the $[[15, 7, 3]]$ code is constructed with a similar parity check matrix, which should look like:

$$H = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

So the entire stabilizer group admits 8 weight-8 stabilizers:

$$S = \begin{pmatrix} X & X & X & X & X & X & X & X & I & I & I & I & I & I & I \\ X & X & X & X & I & I & I & I & X & X & X & X & I & I & I \\ X & X & I & I & X & X & I & I & X & X & I & I & X & X & I \\ X & I & X & I & X & I & X & I & X & I & X & I & X & I & X \\ Z & Z & Z & Z & Z & Z & Z & Z & I & I & I & I & I & I & I \\ Z & Z & Z & Z & I & I & I & I & Z & Z & Z & Z & I & I & I \\ Z & Z & I & I & Z & Z & I & I & Z & Z & I & I & Z & Z & I \\ Z & I & Z & I & Z & I & Z & I & Z & I & Z & I & Z & I & Z \end{pmatrix}$$

1.3 Choosing logical operators

I looked for logical operators that have similar weight to the stabilizers, with the plan of removing two at a time (X and Z pairs) to gauge fix.

For the first two stabilizers

$$\begin{aligned} S_1 &= XXXXXXXXIIIIII \\ S_2 &= XXXXIIIIXXXXII \end{aligned}$$

We can choose to measure $XXXXIIIIIIII$ as part of the stabilizer group, but not require it to be +1 eigenstate, this is a valid logical operator of the code, I will call it G_1 . Then we can measure G_2 and G_3 as shown below in similar way, not requiring +1 eigenstate.

$$\begin{aligned} G_1 &= XXXXIIIIIIII \\ G_2 &= IIIIXXXXIIIIII \\ G_3 &= IIIIIIXXXXII \end{aligned}$$

Then we can recover the stabilizers by

$$\begin{aligned} S_1 &= G_1 G_2 \\ S_2 &= G_1 G_3 \end{aligned}$$

The end effect of this is that we can measure the stabilizers with less total qubit measurements and gates, and we can do this with the other three pairs of stabilizers in the original group, using 4 gauge qubits.

1.4 Scheduling

For the original code, in each noise round I first measure all Z type stabilizers and then all X type stabilizers. In total, this requires 8 time windows each coupling one physical qubit to one of 4 ancilla qubits for a total of 32 gates for each stabilizer type, so 64 gates in total over 16 time windows.

```
12345678
2345    6781
34  67  12  58
4 5 1 2 3 6 7 8
```

For my decomposition idea, there are now 12 stabilizers, each of weight 4:

$$S = \begin{pmatrix} X & X & X & X & I & I & I & I & I & I & I & I & I & I & I \\ I & I & I & I & X & X & X & X & I & I & I & I & I & I & I \\ I & I & I & I & I & I & I & I & X & X & X & X & I & I & I \\ X & I & I & I & X & I & I & I & X & I & I & I & X & I & I \\ I & X & I & I & I & X & I & I & I & X & I & I & I & X & I \\ I & I & X & I & I & I & X & I & I & I & X & I & I & I & X \\ Z & Z & Z & Z & I & I & I & I & I & I & I & I & I & I & I \\ I & I & I & I & Z & Z & Z & Z & I & I & I & I & I & I & I \\ I & I & I & I & I & I & I & I & Z & Z & Z & Z & I & I & I \\ Z & I & I & I & Z & I & I & I & Z & I & I & I & Z & I & I \\ I & Z & I & I & I & Z & I & I & I & Z & I & I & I & Z & I \\ I & I & Z & I & I & I & Z & I & I & I & Z & I & I & I & Z \end{pmatrix}$$

This allows us to couple 6 physical qubits to ancilla per time window, and we can complete all coupling for one stabilizer type in 4 time windows. So in total, we have 24 gates for each stabilizer type, and 48 gates in total.

```
1234
    1234
      1234
2   3   4   1
3   4   1   2
4   1   2   3
```

2 Simulation

I do a memory experiment that involves first some stabilizing rounds to find a proper codespace first. I wanted to try to interleave the X and Z stabilizer data-to-ancilla coupling gates, but ran into the issue that the stabilizer measurements at the end of each noise round were random. I learned that this is due to the interleaving

2.1 Codeword basis

My first run of the memory experiment gave LER that I don't know if it makes any sense. So I investigated the expected behavior by looking at a single noise round and observing the before and after state and whether the syndrome measurement correctly reflects the error.

If I am using the initial state of all 0s or all 1s and measuring logical Z as the xor of all of the physical qubits, then I find that there are 16 superpositions for $|0\rangle_Z$ and $|1\rangle_Z$

$$\begin{aligned}
|0\rangle_Z &= \frac{1}{\sqrt{16}} (|0000000000000000\rangle + |0000111111110000\rangle + |0011001111001100\rangle + |0011110000111100\rangle \\
&\quad + |0101010110101010\rangle + |0101101001011010\rangle + |0110011001100110\rangle + |0110100110010110\rangle \\
&\quad + |1001011010010110\rangle + |1001100101100110\rangle + |1010010101011010\rangle + |1010101010101010\rangle \\
&\quad + |1100001100111100\rangle + |1100110011001100\rangle + |1111000011110000\rangle + |1111111100000000\rangle) \\
|1\rangle_Z &= \frac{1}{\sqrt{16}} (|0000000011111111\rangle + |0000111100001111\rangle + |0011001100110011\rangle + |0011110011000011\rangle \\
&\quad + |0101010101010101\rangle + |0101101010100101\rangle + |0110011010011100\rangle + |0110100101101000\rangle \\
&\quad + |1001011001101000\rangle + |1001100110011100\rangle + |1010010110100101\rangle + |1010101001010101\rangle \\
&\quad + |1100001111000011\rangle + |1100110000110011\rangle + |1111000000001111\rangle + |1111111111111111\rangle)
\end{aligned}$$

2.2 Small scale simulation

My first attempt at the $[[15, 7, 3]]$ code with the Pymatching decoder gave a large LER (0.238), so I investigated the code for a single round to see what was happening.

2.2.1 Single round circuit construction

1. Initialize circuit with all physical qubits in $|0\rangle$
2. Run a "warmup" round to stabilize the code, measuring the 8 stabilizers.
3. In the noise round, I measure the stabilizers again and implement detectors comparing each stabilizer to the previous, which in this case is the warmup. In zero noise, these will be identical and no detectors should fire.
4. With noise, the detectors that fire give the syndrome.

2.2.2 Error injection

I made a modified version of the hamming15 function that has stim error 0 but randomly injects an X and/or Z error to random qubits, which we can track to see the effect on the syndrome measurements and logical readout. Here is the behavior I observe:

- A single error on a physical qubit will be detected properly by syndrome measurement (good) as long as there is no error on the corresponding ancilla.
- Multiple errors of the same type on physical qubits is detected as the sum of the bitstrings on the ancilla qubits, so this fails to detect the error. One x error and one z error can be detected properly by the syndrome measurements (expected).
- Ancilla errors are detrimental and will lead to logical error.

2.2.3 Results with manual decoding

I implemented my own decoding to make sure that it was being done correctly, since I'm not sure exactly how the Pymatching decoder works under the hood. Here is my logic for one round.

1. In a single sample, I sample the circuit with some noise and look at the measurement string. This measurement string includes the 8 ancilla from the warmup, the 8 ancilla from the noise round, and the 15 physical qubits as a result of the logical Z measurement.
2. I replicate the detector function by XORing the two stabilizer measurements (0-7 and 8-15).
3. Then, since I am measuring in logical Z, I am only concerned with the syndrome of the first 4 bits of the detector string, so I find the corresponding index by the logic of the hamming code and flip the bit as indicated (no flip if 0).
4. I then compare this to the most similar codeword to see if it is outside the codespace. I also count logical errors and detector fires. I will call these "logical errors" (LE), "outside codespace" (OC), and "detector fires" (DF).

For first results, I keep track of

- logical errors (LE), shots where the logical Z measurement is 1 (in this specific experiment because we prepare in the $|0\rangle$ state)
- outside codespace (OC), shots where the end measured state is not in the codespace (even after applying error correction)
- detector fires (DF), shots where a detector of the Z type stabilizer fires and thus signifies a successful error detection

2.3 Decomposed Codewords

For the decomposed code, I found there to be 64 codewords, which I think comes from the fact that there are now 6 "stabilizers" per type.